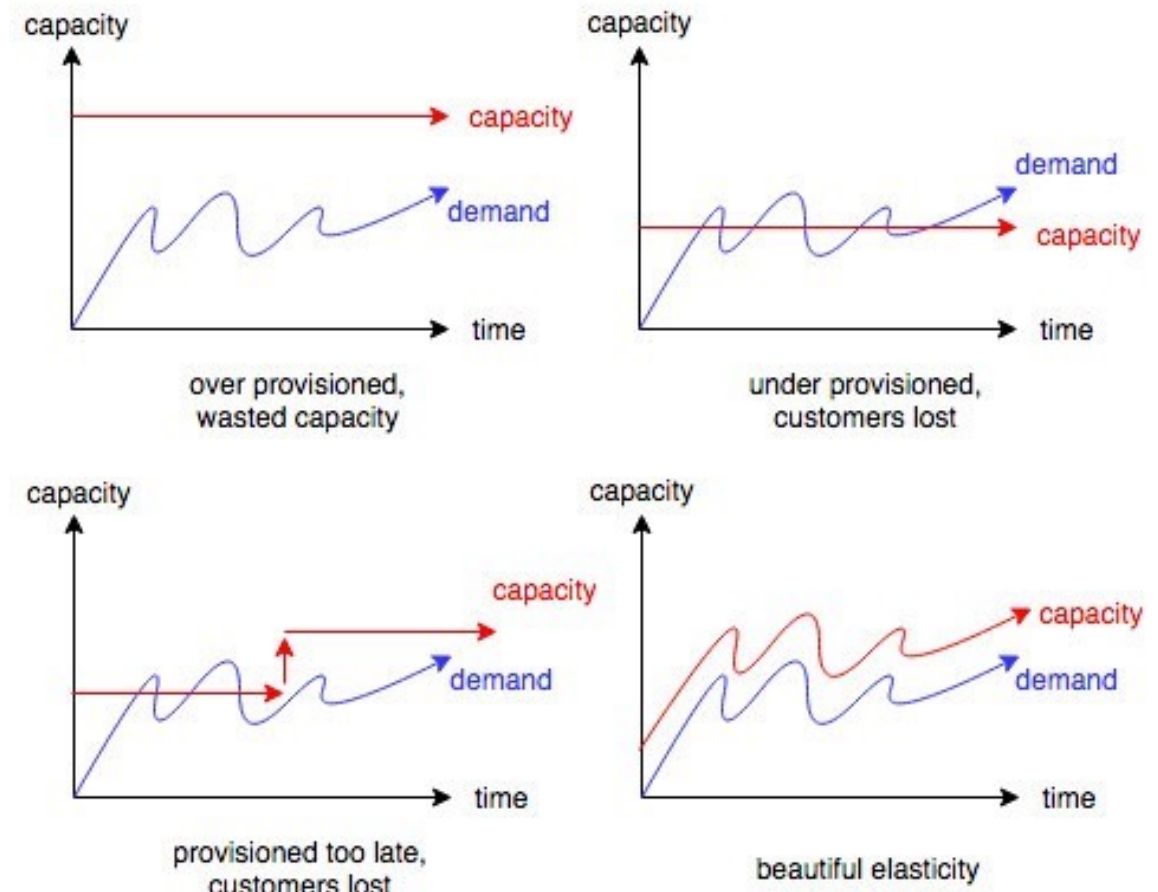
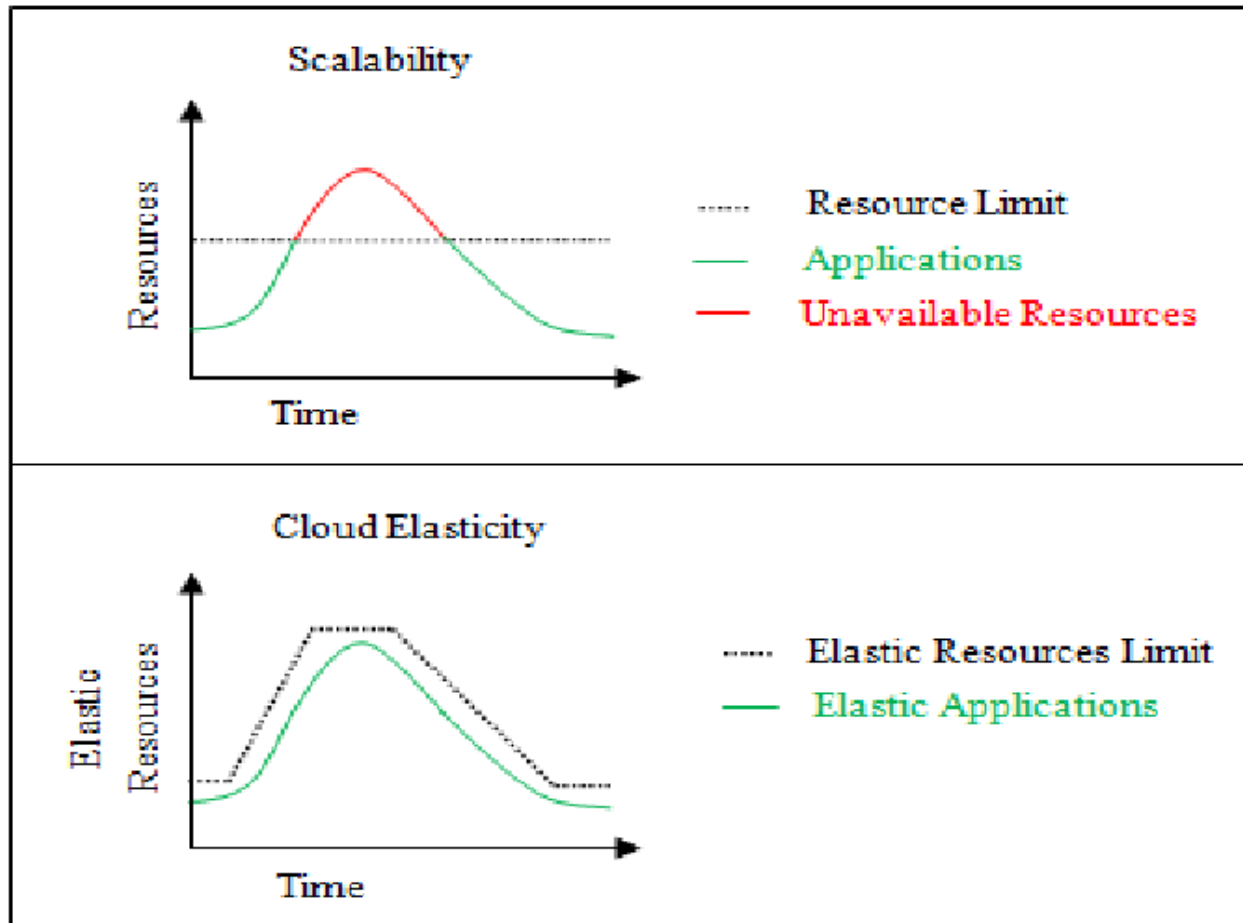


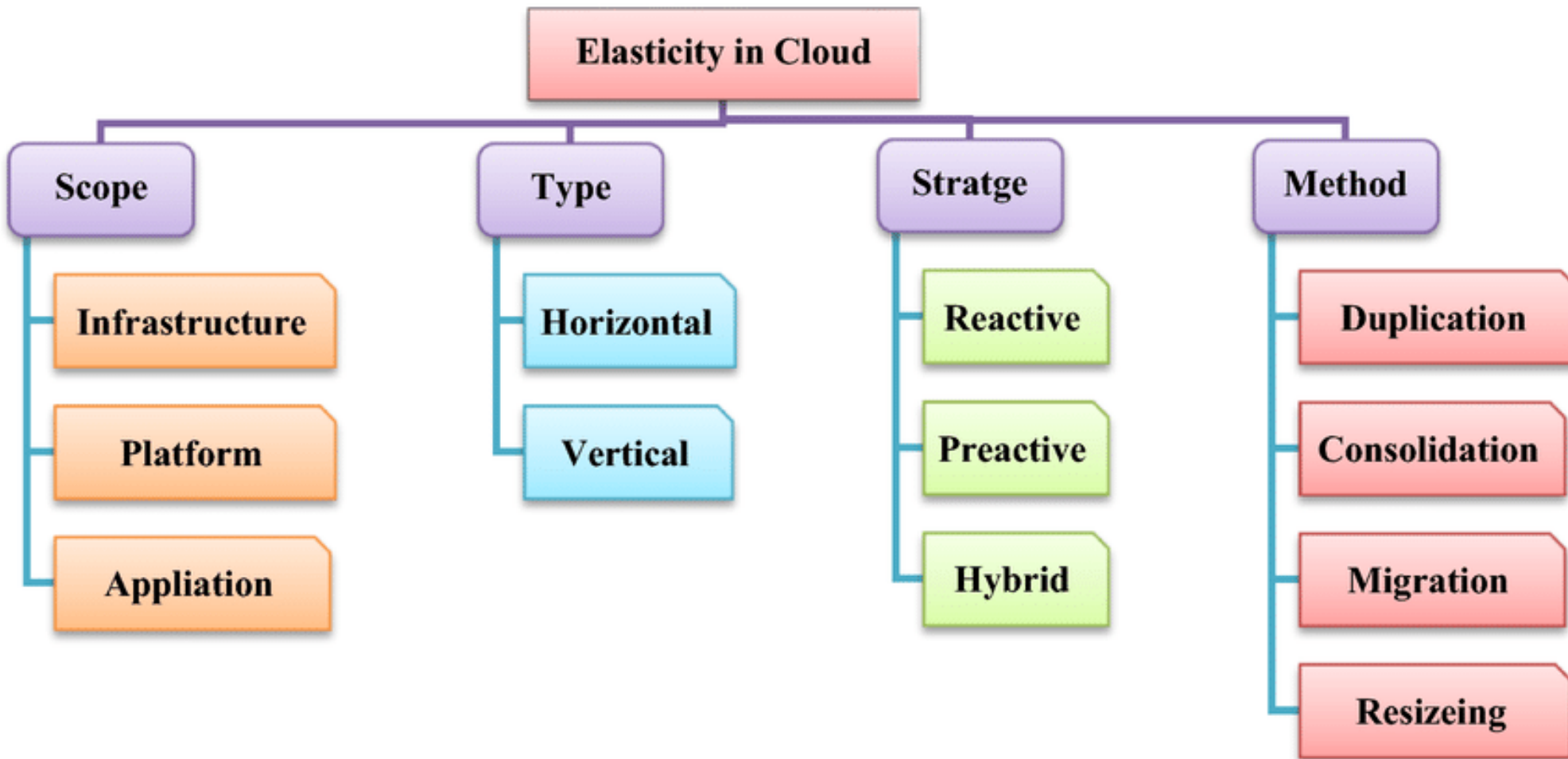
Elasticidade em Nuvem

Por Rodrigo da Rosa Righi

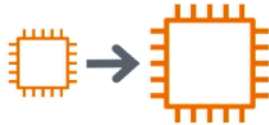
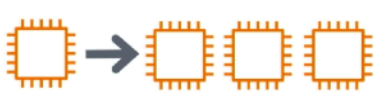
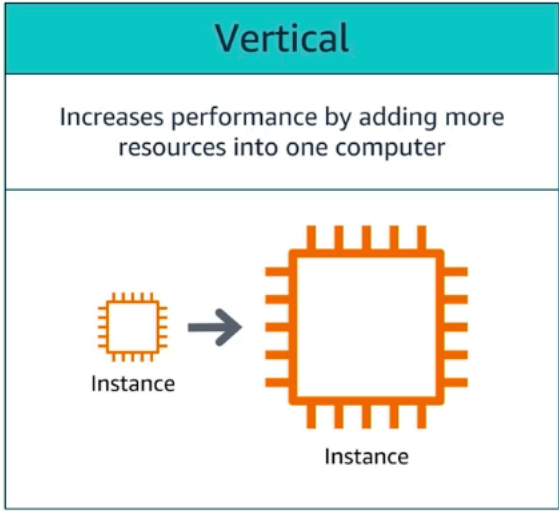
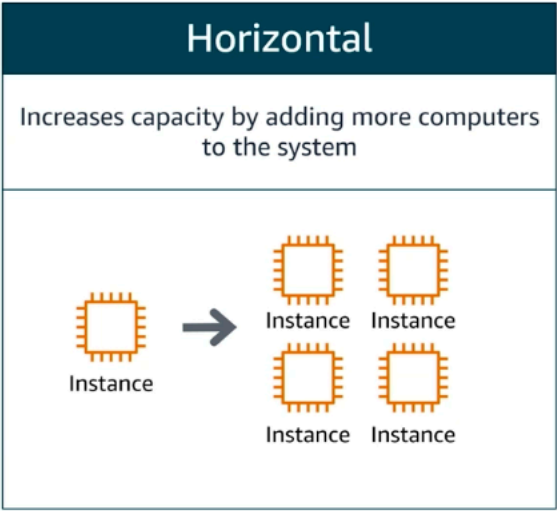
rrrighi@unisinos.br

Figure 1: Scalability Vs Cloud Elasticity





Types of scaling



Horizontal scaling

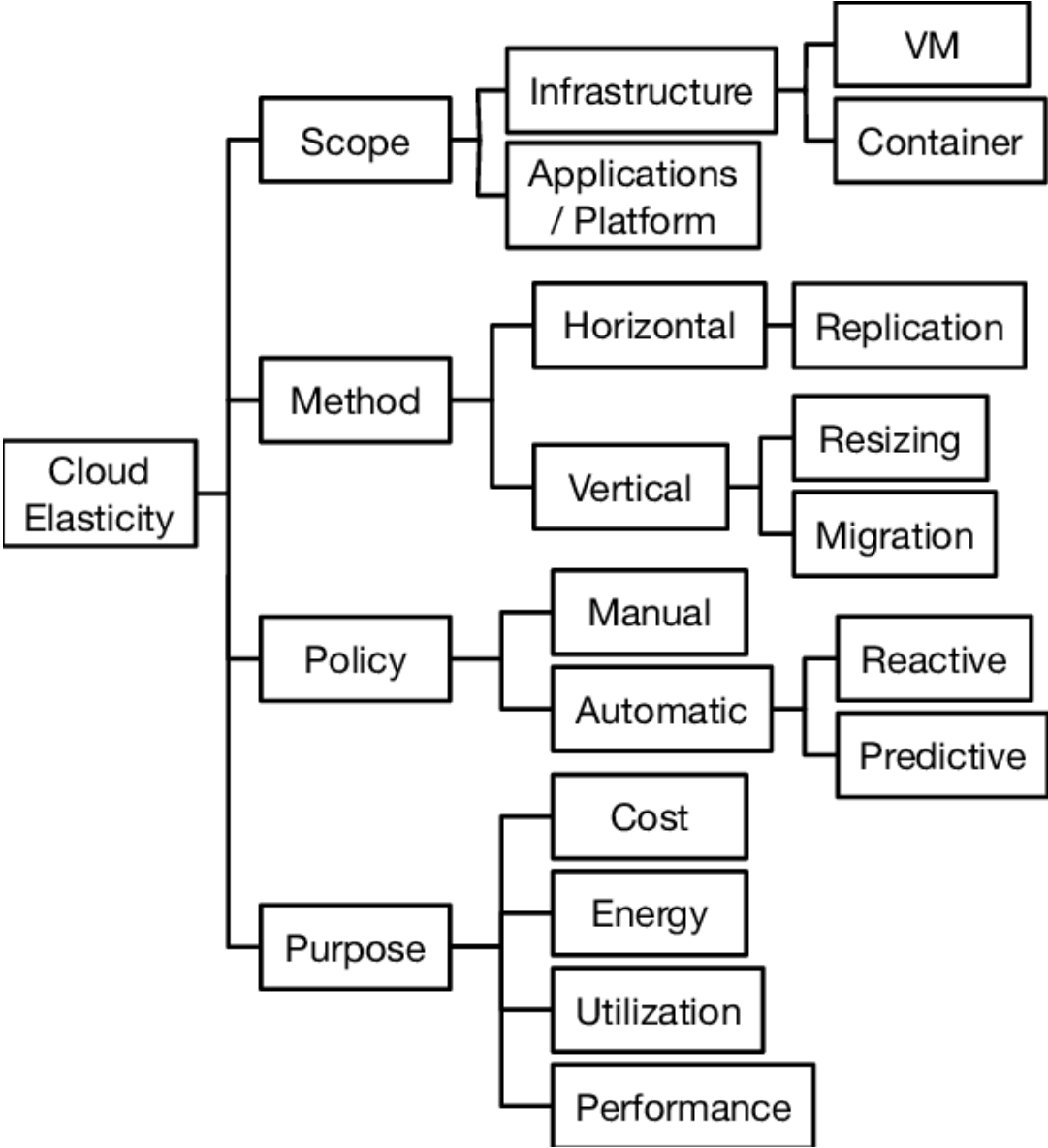
Increase the number of workers that are allocated to the job.

- **Use case:** Working with large, splittable datasets
- **Example:** Processing a large .csv file

Vertical scaling

Choose a worker type with larger CPU, memory, and disk space.

- **Use case:** Working with memory-intensive or disk-intensive applications
- **Example:** Machine learning (ML) transformations



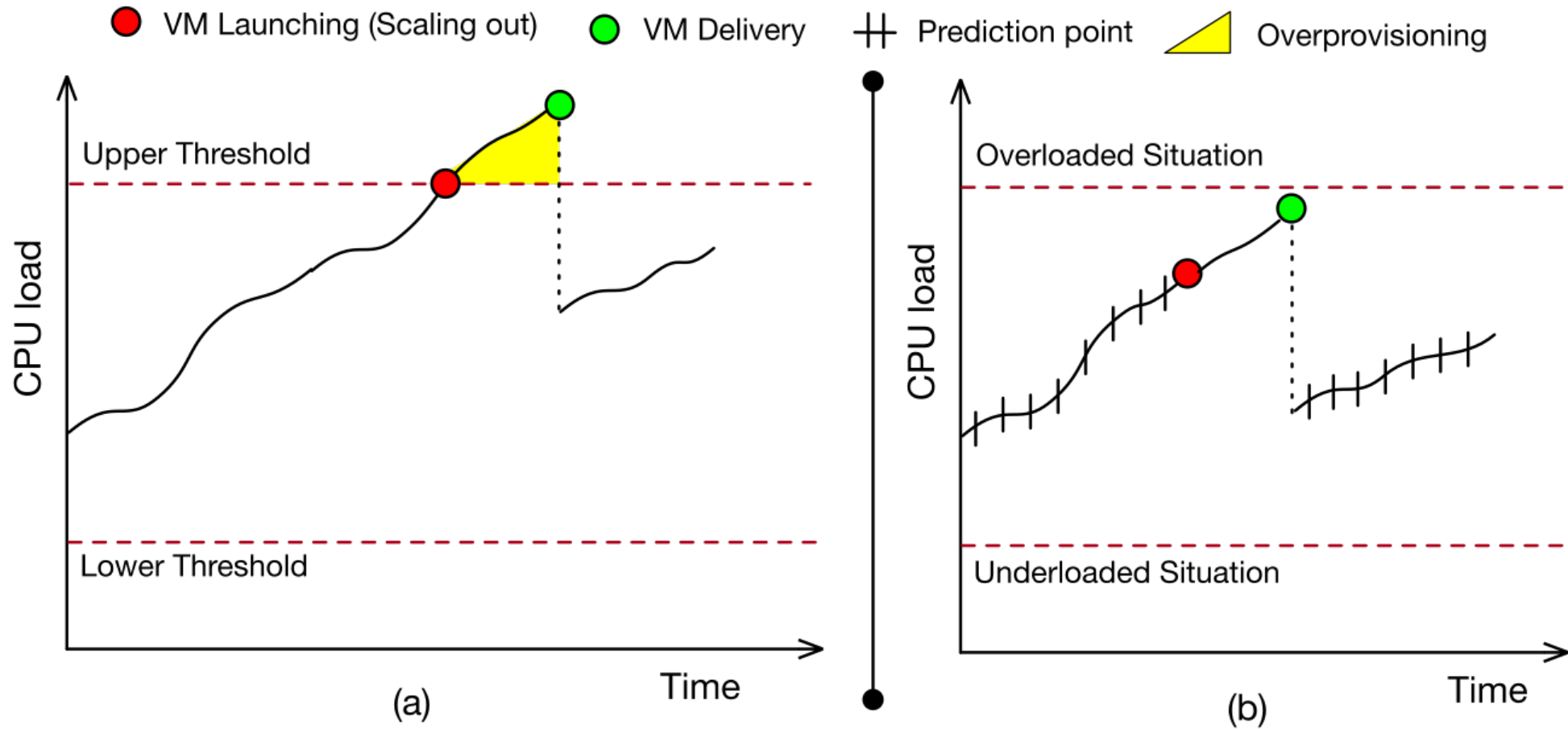
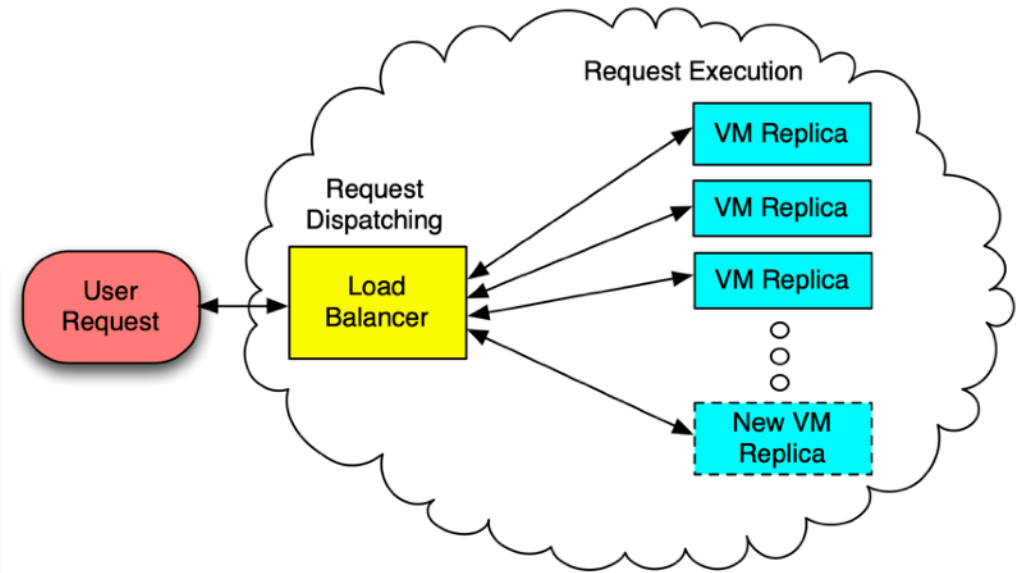


Figure 1 Elasticity approaches: (a) reactive; (b) proactive.

Introduction

The most common approach for **elasticity** is the replication of stand-alone virtual machines (VMs)



This mechanism was originally developed for dynamic scaling **server-based applications**, such as web, e-mail and databases.

Elasticity in the Cloud

```
Template vmtemplate = new Template(templateid, oneClient);
OneResponse rc = vmtemplate.instantiate();
VirtualMachine vm = new VirtualMachine(Integer.parseInt(rc.getMessage()), oneClient);
vm.deploy(hostid);
```

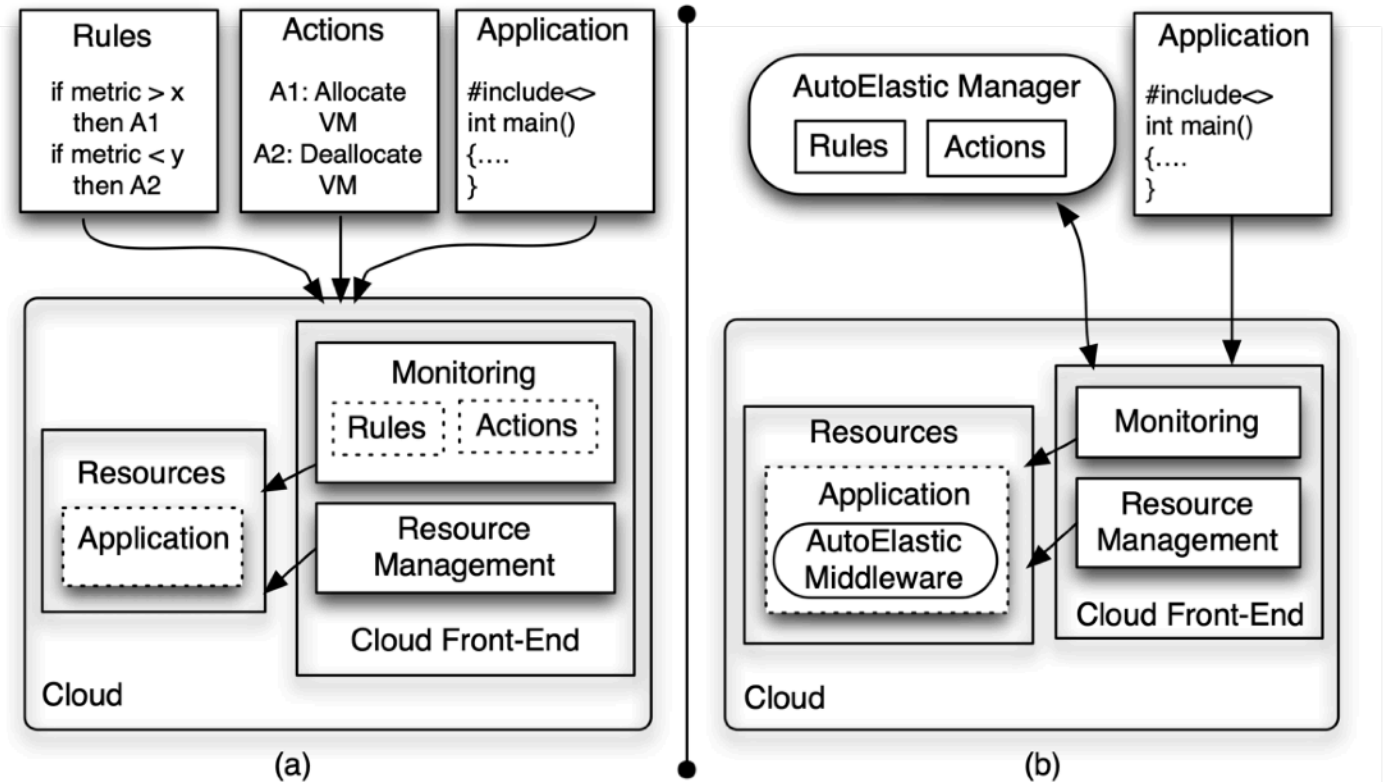
=====

Código do método real que está em produção:

```
public int deploy(Client oc, int hostid, JTextArea plog){
    Template vmtemplate = new Template(this.templateid, oc);
    OneResponse rc = vmtemplate.instantiate();
    log = plog;
    if (!rc.isError()){
        this.id = Integer.parseInt(rc.getMessage());
        this.vm = new VirtualMachine(this.id, oc);
        vm.deploy(hostid);
    }
    return this.id;
}
```

Code to create a new VM

AutoElastic

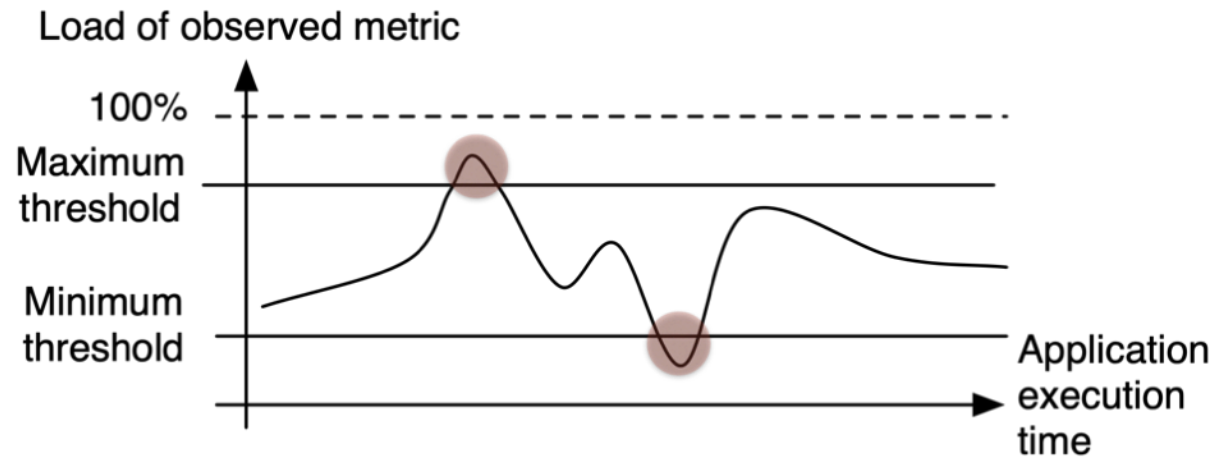


(a) Approach adopted by **Azure** and Amazon **AWS**, in which the **user must define elasticity rules and configure metrics beforehand**; (b) AutoElastic

AutoElastic

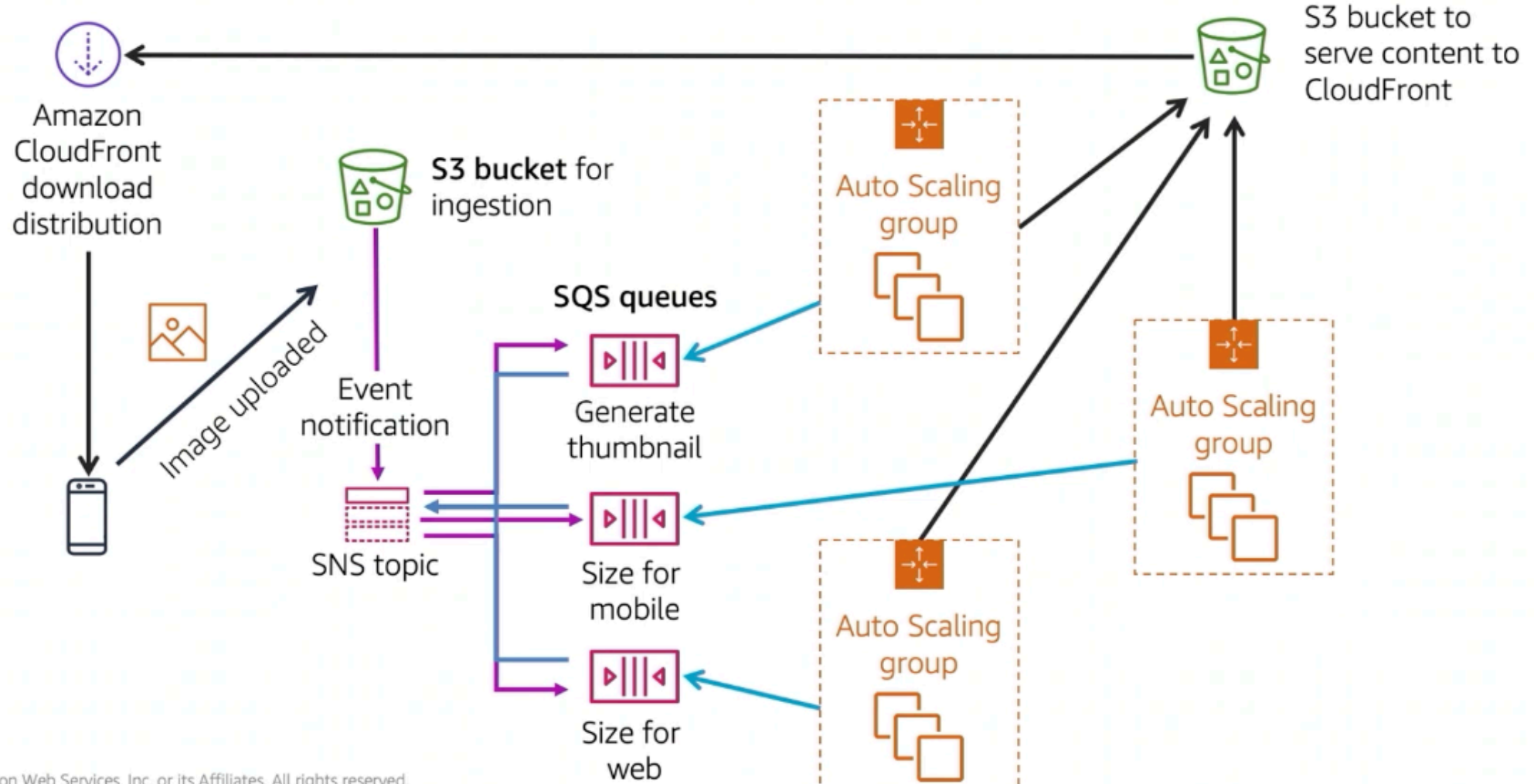
- Values of load: 82, 78, 81, 80, 77, 93:

$$\frac{1}{2} \cdot 93 + \frac{1}{4} \cdot 77 + \frac{1}{8} \cdot 80 + \frac{1}{16} \cdot 81 + \frac{1}{32} \cdot 78 + \frac{1}{64} \cdot 82 = 84.53$$



This scenario illustrates situations where allocation and deallocation actions take place, but they could be neglected for **thrashing avoidance**.

Decoupling example: Using Amazon S3 with Amazon SNS



© 2020 Amazon Web Services, Inc. or its Affiliates. All rights reserved.

Elasticidade em Nuvem

Por Rodrigo da Rosa Righi

rrrighi@unisinos.br